

## LAB 12 | المختبر 12

# Battery Scheduling with Deep Q-Learning

## جدولة بطارية باستخدام DQN

A MATLAB custom environment for time-of-use energy management

بيئة MATLAB مخصصة لإدارة الطاقة وفق التعرفة الزمنية

Prepared by | إعداد: Dr.-Ing. Aouss Gabash | د.م. أوس غباش

### Laboratory Objectives

- Create a custom battery environment with `rlFunctionEnv`.
- Define observations and discrete actions.
- Implement SOC dynamics, cost, degradation, and penalties.
- Train a DQN agent.
- Compare the learned policy with a no-battery baseline.

### أهداف المختبر

- إنشاء بيئة بطارية مخصصة.
- تعريف الملاحظات والأفعال.
- تنفيذ SOC والكلفة والتآكل والعقوبات.
- تدريب وكيل DQN.
- المقارنة مع عدم استخدام البطارية.

## 1. Requirements

- MATLAB
- Reinforcement Learning Toolbox
- Deep Learning Toolbox

## 1. المتطلبات

يلزم MATLAB مع Deep Learning Toolbox و Reinforcement Learning Toolbox.

## 2. Environment

### Observation

SOC, load, price, hour.

### Actions

-1 charge, 0 idle, +1 discharge.

### Episode

24 hourly steps.

### Reward

Negative operating cost and penalties.

## 2. البيئة

تتكون الملاحظة من SOC والحمل والسعر والساعة، والأفعال من الشحن والتوقف والتفريغ، والحلقة يوم واحد.

### 3. Main MATLAB Script

```

%% Lab 12: Battery Scheduling using DQN
clear; clc; close all;
rng(12);

%% Observation and action specifications
obsInfo = rlNumericSpec([4 1], ...
    LowerLimit=[0;0;0;0], ...
    UpperLimit=[1;2;2;1]);
obsInfo.Name = "battery_observation";

actInfo = rlFiniteSetSpec([-1,0,1]);
actInfo.Name = "battery_action";

%% Create custom environment
env = rlFunctionEnv(obsInfo,actInfo, ...
    @batteryStepFunction,@batteryResetFunction);

validateEnvironment(env);

%% Create a default DQN agent
agent = rlDQNAgent(obsInfo,actInfo);

agent.AgentOptions.UseDoubleDQN = true;
agent.AgentOptions.DiscountFactor = 0.99;
agent.AgentOptions.MinibatchSize = 128;
agent.AgentOptions.ExperienceBufferLength = 1e5;
agent.AgentOptions.TargetUpdateFrequency = 4;

agent.AgentOptions.EpsilonGreedyExploration.Epsilon = 1.0;
agent.AgentOptions.EpsilonGreedyExploration.EpsilonMin = 0.05;
agent.AgentOptions.EpsilonGreedyExploration.EpsilonDecay = 1e-4;

%% Training options
trainOpts = rlTrainingOptions( ...
    MaxEpisodes=1200, ...
    MaxStepsPerEpisode=24, ...
    ScoreAveragingWindowLength=50, ...
    StopTrainingCriteria="AverageReward", ...
    StopTrainingValue=-8, ...
    Verbose=false, ...
    Plots="training-progress");

%% Train
trainingStats = train(agent,env,trainOpts);

```

```
%% Simulate one day
simOpts = rlSimulationOptions(MaxSteps=24);
experience = sim(env,agent,simOpts);

obs = experience.Observation.battery_observation.Data;
act = experience.Action.battery_action.Data;
rew = experience.Reward.Data;

soc = squeeze(obs(1,1,:));
actions = squeeze(act);
rewards = squeeze(rew);
hours = 0:numel(actions)-1;

figure;
stairs(0:numel(soc)-1,soc,'LineWidth',1.4);
grid on;
xlabel('Hour'); ylabel('SOC');
title('Battery State of Charge');

figure;
stairs(hours,actions,'LineWidth',1.4);
grid on;
yticks([-1 0 1]);
yticklabels({'Charge','Idle','Discharge'});
xlabel('Hour'); ylabel('Action');
title('DQN Battery Actions');

fprintf('Episode reward = %.3f\n',sum(rewards));
```

### 3. البرنامج الرئيسي

يُنشئ مواصفات الملاحظات والأفعال والبيئة، ثم ينشئ وكيل DQN ويدربه ويشغله ليوم واحد.

## 4. Reset Function

```
function [InitialObservation,Info] = batteryResetFunction()

Info.Load = [2.2 2.0 1.9 1.8 1.9 2.3 ...
             3.0 3.8 4.2 4.0 3.7 3.5 ...
             3.4 3.6 4.0 4.8 5.8 6.5 ...
             6.1 5.4 4.7 4.0 3.2 2.6]';

Info.Price = [0.18*ones(6,1); ...
             0.25*ones(10,1); ...
             0.42*ones(5,1); ...
             0.23*ones(3,1)];

Info.Capacity = 10;          % kWh
Info.Pmax = 2.5;             % kW
Info.EtaCharge = 0.95;
Info.EtaDischarge = 0.95;
Info.SOCmin = 0.10;
Info.SOCmax = 0.90;

Info.Hour = 1;
Info.SOC = 0.50;
Info.TotalCost = 0;

InitialObservation = makeObservation(Info);
end
```

## 4. دالة إعادة الضبط

تهئى الحمل والتعرفة ومعاملات البطارية والساعة وحالة الشحن في بداية كل حلقة.



## 5. Step Function

```

function [NextObservation,Reward,IsDone,Info] = ...
    batteryStepFunction(Action,Info)

action = double(Action);
h = Info.Hour;
loadPower = Info.Load(h);
price = Info.Price(h);

% -1 charge, 0 idle, +1 discharge
PbattCommand = action*Info.Pmax;
socOld = Info.SOC;
violation = 0;

if PbattCommand < 0
    Pcharge = -PbattCommand;
    socNew = socOld + ...
        Info.EtaCharge*Pcharge/Info.Capacity;
elseif PbattCommand > 0
    Pdischarge = PbattCommand;
    socNew = socOld - ...
        Pdischarge/(Info.EtaDischarge*Info.Capacity);
else
    socNew = socOld;
end

if socNew > Info.SOCmax
    violation = socNew-Info.SOCmax;
    socNew = Info.SOCmax;
elseif socNew < Info.SOCmin
    violation = Info.SOCmin-socNew;
    socNew = Info.SOCmin;
end

deltaSOC = socNew-socOld;

if deltaSOC >= 0
    gridBatteryPower = ...
        deltaSOC*Info.Capacity/Info.EtaCharge;
else
    deliveredPower = ...
        -deltaSOC*Info.Capacity*Info.EtaDischarge;
    gridBatteryPower = -deliveredPower;
end

gridPower = max(0,loadPower+gridBatteryPower);

```

```
energyCost = price*gridPower;
degradationCost = 0.015*abs(gridBatteryPower);

Reward = -energyCost ...
         -degradationCost ...
         -30*violation;

Info.SOC = socNew;
Info.TotalCost = Info.TotalCost+energyCost;
Info.Hour = Info.Hour+1;

IsDone = Info.Hour > 24;

if IsDone
    terminalPenalty = 3*abs(Info.SOC-0.50);
    Reward = Reward-terminalPenalty;
    Info.Hour = 24;
end

NextObservation = makeObservation(Info);
end
```

## 5. دالة الخطوة

تحدث حالة الشحن، وتفرض الحدود، وتحسب قدرة الشبكة والكلفة والتآكل والمكافأة.

## 6. Observation Helper

```
function observation = makeObservation(Info)
h = min(Info.Hour,24);

loadNorm = Info.Load(h)/max(Info.Load);
priceNorm = Info.Price(h)/max(Info.Price);
hourNorm = (h-1)/23;

observation = [
    Info.SOC
    loadNorm
    priceNorm
    hourNorm
];
end
```

## 6. بناء الملاحظة

تُطبع قيم الحمل والسعر والساعة وتُجمع مع SOC في متجه من أربعة عناصر.

## 7. Reward

$$r_t = -Cost_t - Degradation_t - 30 \cdot SOCViolation_t$$

A final-SOC penalty prevents the agent from emptying the battery at the end of every episode.

## 7. المكافأة

تعاقب الكلفة والتآكل وتجاوز SOC، وتمنع العقوبة النهائية تفريغ البطارية بالكامل في نهاية اليوم.

## 8. Baseline

```
[~,baseInfo] = batteryResetFunction();
baselineCost = sum(baseInfo.Load.*baseInfo.Price);
fprintf('No-battery daily cost = %.2f EUR\n',baselineCost);
```

Judge the trained policy against a simple baseline, not only by episode reward.

## 8. خط الأساس

احسب كلفة شراء الحمل كاملاً دون بطارية ثم قارنها بسياسة DQN.

## 9. Expected Behavior

- Charge during low-price hours.
- Discharge during expensive evening hours.
- Avoid SOC violations.
- Preserve final SOC.

Training is stochastic. Stopping thresholds may need adjustment when reward scaling changes.

## 9. السلوك المتوقع

يُتوقع الشحن في الساعات الرخيصة والتفريغ في الذروة مع احترام الحدود والمحافظة على SOC نهائية مناسبة.

## 10. Student Experiments

1. Change capacity to 5 and 20 kWh.
2. Use five action levels.
3. Add PV generation.
4. Add a demand-charge penalty.
5. Randomize load and price at reset.
6. Compare with a rule-based controller.

## 10. تجارب الطالب

1. غير سعة البطارية.
2. استخدم خمسة مستويات قدرة.
3. أضف PV.
4. أضف عقوبة للذروة.
5. غير الحمل والسعر عشوائيًا.
6. قارن بتحكم قائم على قواعد.

## 11. Mini Project

**Microgrid extension:** Add PV, a diesel generator, grid-import limits, and load shedding. Balance cost, emissions, reliability, and degradation.

## 11. مشروع مصغر

وسّع البيئة إلى شبكة مصغرة تضم PV ومولد ديزل وحدود استيراد وفصل أحمال.

## 12. Laboratory Report

- Define the MDP.
- Explain SOC dynamics.
- Plot training reward, SOC, and actions.
- Compare cost with the baseline.
- Report violations.
- Discuss reward sensitivity and safety.

## 12. تقرير المخبر

يتضمن تعريف MDP ومعادلات SOC والرسوم والمقارنة مع خط الأساس والمخالفات وتحليل المكافأة والأمان.

## References | المراجع

المراجع العامة المستخدمة في هذا المقرر

[1] A. Gabash, *Flexible Optimal Operations of Energy Supply Networks with Renewable Energy Generation and Battery Storage*. Saarbrücken, Germany: Südwestdeutscher Verlag für Hochschulschriften, 2014.

[2] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th Global ed. Harlow, U.K.: Pearson Education Limited, 2022.

[3] A. Gabash, "Energy Market Transition and Climate Change: A Review of TSOs-DSOs C++ Framework from 1800 to Present," *Energies*, vol. 16, no. 17, Art. no. 6139, 2023, doi: 10.3390/en16176139.

[4] A. Gabash, *AI Applications in Electrical Power Systems*, Version 1.0, 2026. [Online]. Available: <https://aoussgabash.com>

## Publication Information | معلومات النشر

|                  |                                                               |
|------------------|---------------------------------------------------------------|
| Document         | Laboratory 12 · Battery Scheduling with Deep Q-Learning       |
| Course           | AI Applications in Electrical Power Systems                   |
| Author           | Dr.-Ing. Aouss Gabash                                         |
| Version          | 1.0                                                           |
| Publication year | 2026                                                          |
| Official website | <a href="https://aoussgabash.com">https://aoussgabash.com</a> |
| Citation style   | IEEE                                                          |

### Recommended citation:

Gabash, A. (2026). Battery Scheduling with Deep Q-Learning. AI Applications in Electrical Power Systems, Laboratory 12, Version 1.0. <https://aoussgabash.com>



---

المرجع المقترح:

أوس غباش، 2026، مخبر 12، تطبيقات الذكاء الاصطناعي في أنظمة الطاقة الكهربائية، الإصدار 1.0، <https://aoussgabash.com>

---

© 2026 Dr.-Ing. Aouss Gabash. Educational use with attribution to the author and official website.